

Reading an Item from Azure Cosmos DB Using the .NET SDK

Summary:-

In this lab, you will use the Azure Cosmos DB .NET SDK to retrieve a specific document from an Azure Cosmos DB container. You will read an item by providing its **Item ID** and **Partition Key**, and display the retrieved document details in the console.

Let's Start:-

After completing this lab, you will be able to:

- Read a document from an Azure Cosmos DB container using the .NET SDK.
- Retrieve an item using its **Item ID** and **Partition Key**.
- Access the returned resource from an ItemResponse.
- Display document properties in the console.

Lab Steps:-

Step 1 – Prepare the Azure Cosmos DB .NET Project

- **Reference:** This lab uses the .NET console application, Azure Cosmos DB account, database, and container created in the previous lab.
- For that **create** Azure Cosmos DB account by selecting **Azure Cosmos DB for NoSQL** from the recommended APIs.
- Configure:
Workload type: Learning, **Region:** EAST US/ EAST US 2, **Capacity Mode:** Provisioned Throughput
- Enable **Apply Free Tier Discount**. Click **Next**.
- Ensure Geo-Redundancy and Multi-region Writes is **Disabled**. Click **Next**.
- Select **Enable access from all networks**. Click on **Review + Create** to create.
- From the left pane, select **Keys**.
- Copy: **URI** and **Primary Key**
- Open **Visual Studio Code**. Create a new project folder.
- Press **Ctrl + Shift + P** to open the **Command Palette**.
- Select **Show and Run Commands** > **.NET: New Project** > **Console App**(Rename it as cosmosdb-app) > **Default Directory** > **.slnx** > **Create Project**.
- **Open** the integrated terminal. Run:

```
cd .\cosmosdb-app\  
dotnet add package Microsoft.Azure.Cosmos  
dotnet add package Newtonsoft.Json
```
- Create a new class file named **Order.cs**.
- Open the following project files:

Program.cs

Order.cs

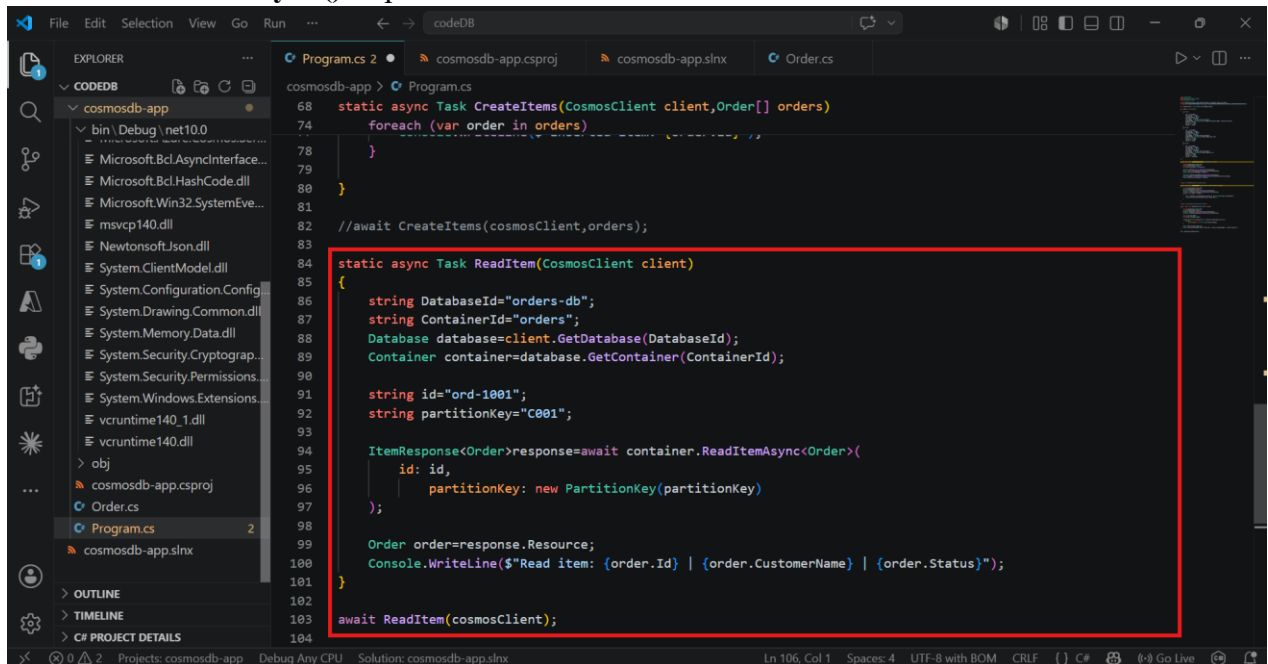
cosmosdb-app.csproj

cosmosdb-app.slnx

- Replace the existing content with the **provided files**.
- In **Program.cs**, **Replace** Endpoint with **URI** and Key with **Primary Key**.
- **Save** all the files and **run**:
dotnet run
dotnet build
- This step creates a .NET console application and prepares it to insert items into the Azure Cosmos DB container.

Step 2 – Add the Read Item Code

- Open **Program.cs**.
- Comment out the previous method call used to insert items.
- Replace the existing code with the provided **Program.cs** file containing the **ReadItemAsync()** implementation.



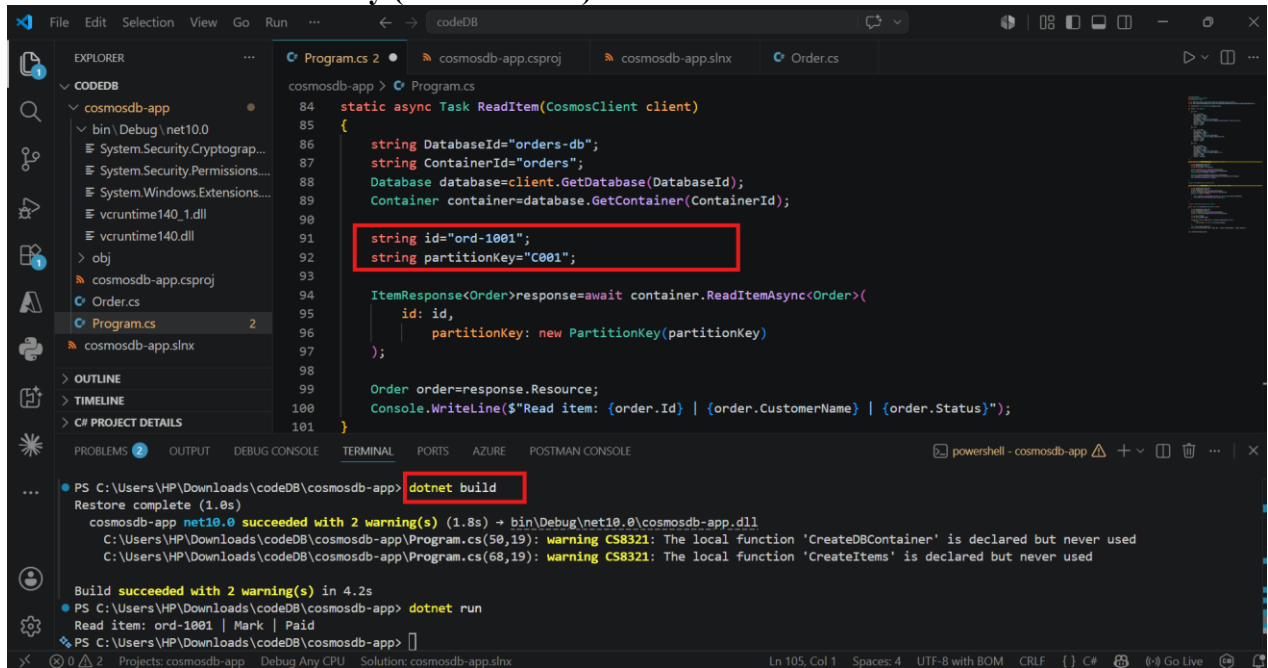
```
68 static async Task CreateItems(CosmosClient client, Order[] orders)
74     foreach (var order in orders)
78     {
79     }
80 }
81
82 //await CreateItems(cosmosClient, orders);
83
84 static async Task ReadItem(CosmosClient client)
85 {
86     string DatabaseId="orders-db";
87     string ContainerId="orders";
88     Database database=client.GetDatabase(DatabaseId);
89     Container container=database.GetContainer(ContainerId);
90
91     string id="ord-1001";
92     string partitionKey="C001";
93
94     ItemResponse<Order>response=await container.ReadItemAsync<Order>({
95         id: id,
96         partitionKey: new PartitionKey(partitionKey)
97     });
98
99     Order order=response.Resource;
100     Console.WriteLine($"Read item: {order.Id} | {order.CustomerName} | {order.Status}");
101 }
102
103 await ReadItem(cosmosClient);
104
```

- **Save** the file and **Run**:
dotnet build
dotnet run
- This step adds the code required to retrieve an item from Azure Cosmos DB.

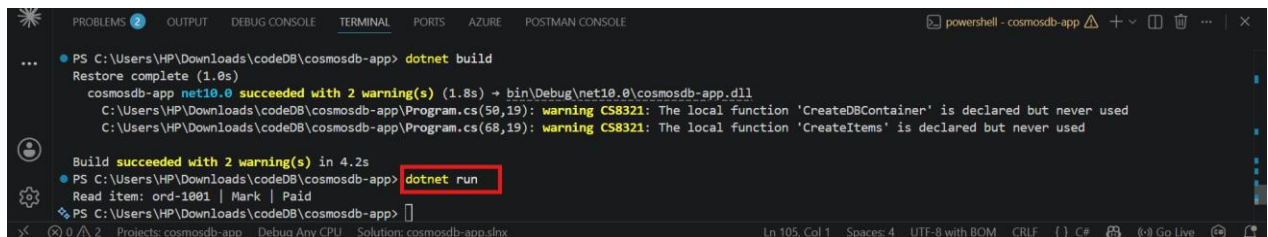
Step 3 – Read an Item

- In the code, specify:
 - **Item ID:** ord-1001

- **Partition Key (Customer ID): C001**



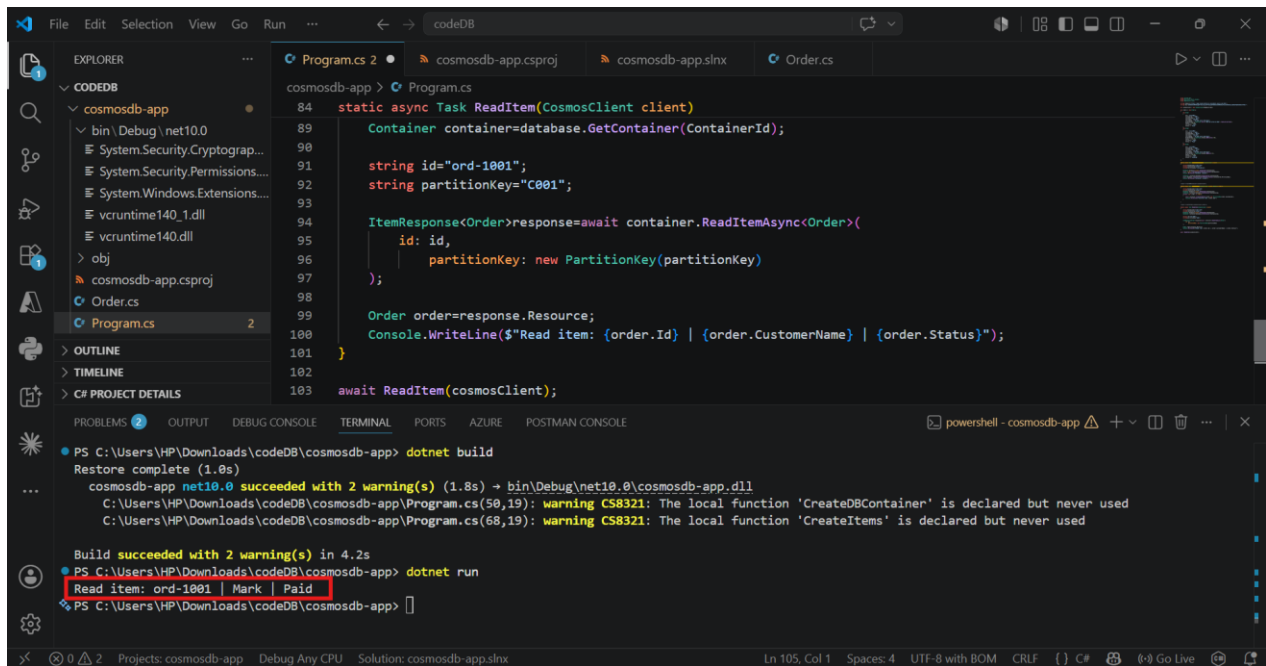
- **Build** the project by using:
dotnet build
- **Run** the application by using:
dotnet run



- This step reads the specified document using its Item ID and Partition Key.

Step 4 – View the Retrieved Item

- Observe the **console output** and verify that the application displays:
 - Order ID
 - Customer Name
 - Status



- This step displays the retrieved document details in the console.

Verification

Verify the following:

- The application connects successfully to Azure Cosmos DB.
- The specified document is retrieved successfully.
- The correct **Order ID**, **Customer Name**, and **Status** are displayed.
- The application executes without any errors.